

COMP3308/3608 Week 10 - Support Vector Machines, Ensembles of Classifiers

Author
K. Yu
2026

Contents

1	Support Vector Machines	2
1.1	Main Idea	2
1.2	Margin and Support Vectors	2
1.3	Linear SVM with Hard Margin	3
1.4	Dual formulation	5
1.5	Linear SVM with Soft-Margin	6
1.6	Nonlinear SVM	8
1.7	Kernel Trick	9
1.8	Common kernel functions	10
1.9	Linear SVM vs Nonlinear SVM	11
1.10	SVM and Generalisation	11
1.11	Comparison with Perceptrons and Neural Networks	12
1.12	Multiclass classification	13
2	Ensembles of Classifiers	14
2.1	Ensemble Learning	14
2.2	Why ensembles work	14
2.3	Methods for creating Ensembles	15
2.4	Bagging	15
2.5	Boosting	17
2.6	AdaBoost	17
2.7	Gradient Boosting (Not Examinable)	19
2.8	Bagging vs Boosting	21
2.9	Random Forest	21

1 Support Vector Machines

1.1 Main Idea

A **Support Vector Machine** (SVM) is a supervised learning method mainly used for classification.

For binary classification, the training set is $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$, $y^{(i)} \in \{-1, 1\}$. The goal is to learn a decision boundary that separates the two classes.

For a linear SVM, the decision boundary is a hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$, where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ is the bias term, and $\mathbf{x}$ is the feature vector. A new example $\mathbf{z}$ is classified by

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b).$$

If $\mathbf{w}^\top \mathbf{x} + b > 0$, then the example is predicted as class 1. If $\mathbf{w}^\top \mathbf{x} + b < 0$, then the example is predicted as class -1 .

Why SVM is different from ordinary linear classifiers

For linearly separable data, there may be many possible separating hyperplanes. SVM does not just choose any separating hyperplane. It chooses the one with the **largest margin**. The **margin** is the distance between the decision boundary and the closest training examples from either class. The SVM decision boundary is therefore called the **maximum margin hyperplane**.

The intuition is simple:

$$\text{larger margin} \implies \text{less sensitive to noise} \implies \text{better generalisation}.$$

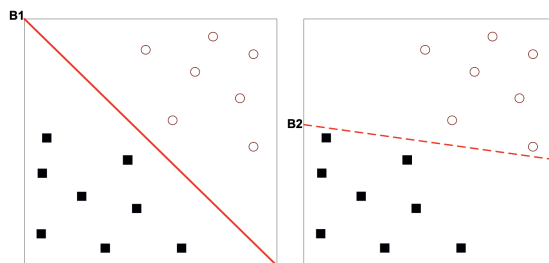


Figure 1: A comparison between two decision boundaries. Both may separate the training data, but the wider-margin boundary is preferred.

1.2 Margin and Support Vectors

Margin

Suppose the decision boundary H is $\mathbf{w}^\top \mathbf{x} + b = 0$. SVM defines two parallel hyperplanes around it: H_1 , $\mathbf{w}^\top \mathbf{x} + b = 1$, and H_2 , $\mathbf{w}^\top \mathbf{x} + b = -1$. The decision boundary H lies exactly in the middle

of H_1 and H_2 . It can be shown that

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}.$$

Therefore, maximising the margin is equivalent to minimising $\|\mathbf{w}\|$. In optimisation, it is more convenient to minimise $\frac{1}{2}\|\mathbf{w}\|^2$. This gives the same solution because $\frac{1}{2}\|\mathbf{w}\|^2$ is a monotonic transformation of $\|\mathbf{w}\|$ for non-negative values.

Support vectors

The **support vectors** are the training examples closest to the decision boundary. They lie on the margin hyperplanes $\mathbf{w}^\top \mathbf{x}^{(i)} + b = 1$ or $\mathbf{w}^\top \mathbf{x}^{(i)} + b = -1$. Equivalently, for support vectors:

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1.$$

Support vectors are important because they define the decision boundary. If a support vector moves, the decision boundary may change. If a non-support vector moves slightly, the decision boundary usually does not change. This is why the method is called SVM: the boundary is supported by a subset of the training vectors.

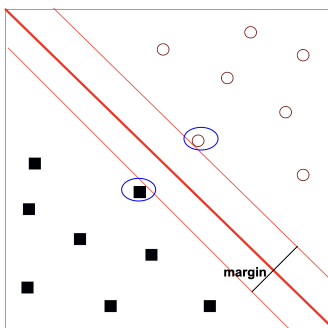


Figure 2: The circled points around the margin are the support vectors. Note that it is possible to have more than 1 support vector for each class

1.3 Linear SVM with Hard Margin

Hard-margin assumption

A **hard-margin SVM** assumes the data is linearly separable. This means there exists a hyper-plane such that all possible examples are on one side and all negative examples are on the other side. For every training example $(\mathbf{x}^{(i)}, y^{(i)})$, $i = 1, \dots, n$, we require correct classification with margin:

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1.$$

This single compact constraint combines two cases.

1. For positive examples, $y^{(i)} = 1$: $\mathbf{w}^\top \mathbf{x}^{(i)} + b \geq 1$.
2. For negative examples, $y^{(i)} = -1$: $\mathbf{w}^\top \mathbf{x}^{(i)} + b \leq -1$.

So the constraint

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1$$

means every training example is correctly classified and lies outside or on the margin.

Primal optimisation problem

The hard-margin SVM learns the maximum-margin hyperplane by solving:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \dots, n.$$

The objective $\frac{1}{2} \|\mathbf{w}\|^2$ is minimised because this maximises the margin $\frac{2}{\|\mathbf{w}\|}$. The constraints ensure that all training examples are correctly classified.

So the hard-margin SVM balances two requirements:

maximise margin + classify all training examples correctly.

Actually, in hard-margin SVM, there is no trade-off yet. Both requirements must be satisfied.

Geometric interpretation

The hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is the decision boundary. The margin hyperplanes are $\mathbf{w}^\top \mathbf{x} + b = 1$ and $\mathbf{w}^\top \mathbf{x} + b = -1$.

Training examples from class 1 must lie on or above H_1 :

$$\mathbf{w}^\top \mathbf{x}^{(i)} + b \geq 1.$$

Training examples from class -1 must lie on or below H_2 :

$$\mathbf{w}^\top \mathbf{x}^{(i)} + b \leq -1.$$

The region between H_1 and H_2 is the margin band. No training example is allowed inside the margin for hard-margin SVM.

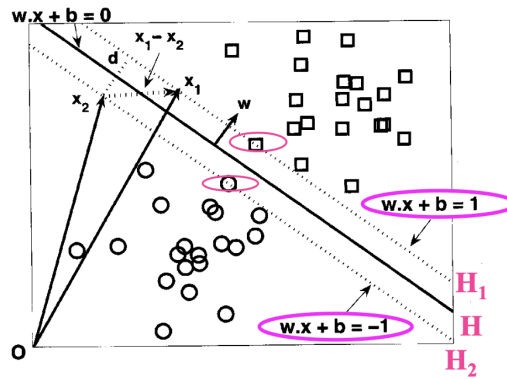


Figure 3: Geometrical view of H , H_1 , H_2 , support vectors, and margin $d = \frac{2}{\|\mathbf{w}\|}$.

1.4 Dual formulation

Why use the dual form?

The primal problem uses $\mathbf{w}$ and b directly. However, SVM is usually solved through its **dual formulation**, because the dual form expresses the solution using inner products between training example, reveals the role of support vectors and allows the kernel trick for nonlinear SVMs. The dual form introduces a Lagrange multiplier λ_i for each training example.

Dual optimisation problem

The dual hard-margin SVM problem is:

$$\begin{aligned} \max_{\boldsymbol{\lambda}} \quad & \left[\sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y^{(i)} y^{(j)} (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)} \right] \\ \text{s.t. } & \lambda_i \geq 0, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \lambda_i y^{(i)} = 0. \end{aligned}$$

Here, $\boldsymbol{\lambda} = [\lambda_1 \ \dots \ \lambda_n]^\top$ is the vector of Lagrange multipliers for each example. The optimisation is a quadratic programming (QP) problem. After solving for $\boldsymbol{\lambda}$, the weights are:

$$\mathbf{w} = \sum_{i=1}^n \lambda_i y^{(i)} \mathbf{x}^{(i)}.$$

Support vectors in the dual form

Many of the λ_i 's are zero. If $\lambda_i = 0$, then $\mathbf{x}^{(i)}$ does not directly affect the decision boundary. If $\lambda_i > 0$, then $\mathbf{x}^{(i)}$ is a support vector. So the weight vector can be written as a sum over support vectors only

$$\mathbf{w} = \sum_{i \in SV} \lambda_i y^{(i)} \mathbf{x}^{(i)}.$$

Here SV is the set of support vector indices.

This is one of the key ideas of SVM:

The decision boundary depends only on support vectors.

Classification using the dual form

For a new example $\mathbf{z}$, the classifier is

$$f(\mathbf{z}) = \mathbf{w}^\top \mathbf{z} + b.$$

Substituting $\mathbf{w} = \sum_{i=1}^n \lambda_i y^{(i)} \mathbf{x}^{(i)}$, we get

$$f(\mathbf{z}) = \sum_{i=1}^n \lambda_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{z} + b.$$

Since only support vectors have non-zero λ_i , this is usually written as:

$$f(\mathbf{z}) = \sum_{i \in SV} \lambda_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{z} + b.$$

The predicted class is:

$$\hat{y} = \text{sign } f(\mathbf{z}).$$

So, classification depends on inner products between the new example $\mathbf{z}$ and the support vectors.

1.5 Linear SVM with Soft-Margin

Motivation

Hard-margin SVM requires perfect linear separability. In practice, this is often unrealistic because:

- data may contain noise;
- examples may overlap;
- some labels may be wrong;
- a perfect separator may overfit.

A **soft-margin SVM** allows some margin violations and even some misclassifications.

The key idea is to trade off **large margin** against **small training error**. This often gives better generalisation than forcing every training example to be classified perfectly.

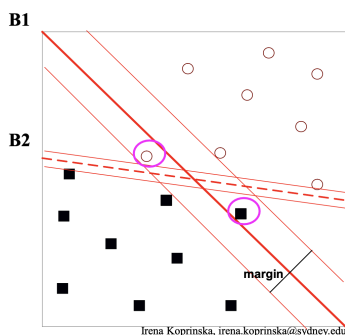


Figure 4: B_1 has a larger margin but misclassifies two new examples, while B_2 classifies them correctly. But B_1 may still be preferred because the new examples may be noise.

Slack variables

Soft-margin SVM introduces slack variables $\xi_i \geq 0$, $i = 1, \dots, n$. The constraint becomes

$$y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i.$$

The slack variable ξ_i , measures how much example i violates the margin.

Interpretation:

- $\xi_i = 0$: correctly classified and outside or on the margin;
- $0 < \xi_i < 1$: correctly classified but inside the margin;
- $\xi_i \geq 1$: misclassified or on the wrong side of the decision boundary.

Soft-margin primal problem

The soft-margin SVM solves:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} & \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right) \\ \text{s.t. } & y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, n, \\ & \xi_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

Here $C > 0$ is a hyperparameter. The objective has two parts:

- $\frac{1}{2} \|\mathbf{w}\|^2$ encourages a large margin.
- $C \sum_{i=1}^n \xi_i$ penalises margin violations.

Meaning of C

The parameter C controls the trade-off between margin size and training errors. If C is large, margin violations are heavily penalised. The classifier tries hard to classify the training examples correctly. This may lead to a narrower margin and possible overfitting. If C is small, margin violations are less strongly penalised. The classifier allows more violations in exchange for a wider margin. This may improve generalisation but can underfit if C is too small.

Soft-margin dual form

In the dual form, the soft-margin solution is similar to the hard-margin solution, except the Lagrange multipliers now have an upper bound $0 \leq \lambda_i \leq C$. The dual optimisation problem becomes:

$$\begin{aligned} \max_{\lambda} & \left[\sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y^{(i)} y^{(j)} (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)} \right] \\ \text{s.t. } & 0 \leq \lambda_i \leq C, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \lambda_i y^{(i)} = 0. \end{aligned}$$

The decision function is still:

$$f(\mathbf{z}) = \sum_{i \in SV} \lambda_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{z} + b.$$

The key difference from hard-margin SVM is $\lambda_i \leq C$.

1.6 Nonlinear SVM

Motivation

Many real-world datasets are not linearly separable in the original feature space. A soft-margin linear SVM can still find a linear boundary, but sometimes the true decision boundary is nonlinear. For example, one class may surround another class. No straight line can separate them well in the original space.

The main idea of nonlinear SVM is to map the data into a higher-dimensional feature space where a linear separator may exist.

Feature mapping

Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ be a feature transformation, where $D > d$. We then learn a linear SVM in the transformed space:

$$\mathbf{w}^\top \phi(\mathbf{x}) + b = 0.$$

This is linear in the transformed space but may correspond to a nonlinear boundary in the original space.

Example 1.6.1. Suppose $\mathbf{x} = [x_1 \ x_2]^\top \in \mathbb{R}^2$. Define

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1^3 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix} \in \mathbb{R}^3.$$

This mapping creates nonlinear features from the original input. A linear hyperplane in the transformed three-dimensional space may correspond to a curved decision boundary in the original two-dimensional space.

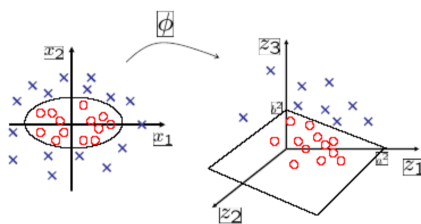


Figure 5: Nonlinearly separable data in $\mathbb{R}^2$ being mapped to $\mathbb{R}^3$ by $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ where it becomes linearly separable.

Cover's theorem intuition

Cover's theorem gives the motivation for this transformation.

Theorem 1.6.1 (Cover's theorem, informal). *A complex classification problem is more likely to be linearly separable in a higher-dimensional feature space.*

This motivates two properties of the transformation:

1. it should be nonlinear;
2. it should map the data to a higher-dimensional space.

However, mapping to a very high-dimensional space also creates a risk of overfitting. SVM controls this risk by still choosing the maximum-margin hyperplane in the transformed space.

1.7 Kernel Trick

The computational problem

The dual SVM formulation only depends on inner products between training examples $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle = (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}$. If we transform the data using ϕ , then the dual formulation depends on $\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)})$. The problem is that $\phi(\mathbf{x})$ may be very high-dimensional, or even infinite-dimensional. Explicitly computing $\phi(\mathbf{x})$ may be computationally expensive. The **kernel trick** solves this problem.

Definition 1.7.1 (Kernel function). A kernel function is a function $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$K(\mathbf{u}, \mathbf{v}) = \phi(\mathbf{u})^\top \phi(\mathbf{v}).$$

This means the kernel computes the inner product in the transformed feature space without explicitly computing the transformed vectors.

Example 1.7.1. Consider the mapping defined in Example 1.6.1. Let $\mathbf{u} = [u_1 \ u_2]^\top$, $\mathbf{v} = [v_1 \ v_2]^\top$. Then

$$\phi(\mathbf{u}) = \begin{bmatrix} u_1^2 \\ \sqrt{2}u_1u_2 \\ u_2^2 \end{bmatrix}, \quad \phi(\mathbf{v}) = \begin{bmatrix} v_1^2 \\ \sqrt{2}v_1v_2 \\ v_2^2 \end{bmatrix}.$$

Their inner product is

$$\phi(\mathbf{u})^\top \phi(\mathbf{v}) = u_1^2v_1^2 + 2u_1u_2v_1v_2 + u_2^2v_2^2 = (u_1v_1 + u_2v_2)^2.$$

But

$$u_1v_1 + u_2v_2 = \mathbf{u}^\top \mathbf{v}.$$

Therefore,

$$\phi(\mathbf{u})^\top \phi(\mathbf{v}) = (\mathbf{u}^\top \mathbf{v})^2.$$

So the kernel is:

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u}^\top \mathbf{v})^2.$$

This means we can compute the inner product in the three-dimensional transformed space using only the inner product in the original two-dimensional space.

SVM training with kernels

The kernelised dual objective is:

$$\begin{aligned} \max_{\boldsymbol{\lambda}} \quad & \left[\sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y^{(i)} y^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \right] \\ \text{s.t. } & 0 \leq \lambda_i \leq C, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \lambda_i y^{(i)} = 0. \end{aligned}$$

For hard-margin SVM, the constraint is instead $\lambda_i \geq 0$. For soft-margin SVM, $0 \leq \lambda_i \leq C$.

Classification with kernels

For a new example $\mathbf{z}$, the nonlinear SVM decision function is

$$f(\mathbf{z}) = \sum_{i \in SV} \lambda_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{z}) + b.$$

The prediction is

$$\hat{y} = \text{sign } f(\mathbf{z}).$$

1.8 Common kernel functions

Polynomial kernel

The polynomial kernel has the form

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u}^\top \mathbf{v} + c)^p,$$

where p is the polynomial degree, c is a constant, often $c = 1$.

RBF kernel

The Radial Basis Function kernel, also called the Gaussian kernel, is

$$K(\mathbf{u}, \mathbf{v}) = \exp \left(- \frac{\|\mathbf{u} - \mathbf{v}\|^2}{2\sigma^2} \right).$$

The RBF kernel measures similarity based on distance. If $\mathbf{u}$ and $\mathbf{v}$ are close, then $K(\mathbf{u}, \mathbf{v})$ is close to 1. If they are far apart, then $K(\mathbf{u}, \mathbf{v})$ is close to 0. The parameter σ controls how quickly similarity decreases with distance. A small σ gives very local influence and can lead to complex boundaries. A large σ gives smoother boundaries.

Sigmoid kernel

The sigmoid kernel is

$$K(\mathbf{u}, \mathbf{v}) = \tanh(k\mathbf{u}^\top \mathbf{v} - \theta).$$

This kernel is related to neural networks, but it only satisfies Mercer's condition for some values of k and θ .

In practice, the most commonly used kernels are usually linear, polynomial and RBF kernel.

Mercer's theorem

Not every function can be used as a valid kernel.

Theorem 1.8.1 (Mercer's theorem). *For a function K to be a valid kernel, it must correspond to some feature mapping ϕ such that*

$$K(\mathbf{u}, \mathbf{v}) = \phi(\mathbf{u})^\top \phi(\mathbf{v}).$$

The main idea is that a valid kernel behaves like an inner product in some feature space.

1.9 Linear SVM vs Nonlinear SVM

Linear SVM

A linear SVM uses no nonlinear kernel. It learns a linear decision boundary in the original feature space $\mathbf{w}^\top \mathbf{x} + b = 0$. The decision function is

$$f(\mathbf{z}) = \sum_{i \in SV} \lambda_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{z} + b.$$

This is useful when the classes are approximately linearly separable in the original feature space.

Nonlinear SVM

A nonlinear SVM uses a kernel function. It learns a linear decision boundary in a transformed feature space $\mathbf{w}^\top \phi(\mathbf{x}) + b = 0$. But in the original feature space, this corresponds to a nonlinear decision boundary. The decision function is

$$f(\mathbf{z}) = \sum_{i \in SV} \lambda_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{z}) + b.$$

1.10 SVM and Generalisation

Why maximum margin helps

The intuitive justification is that, if the boundary is far from the training points, small perturbations are less likely to cause misclassification. So a large-margin classifier is more robust to noise.

Generalisation error depends on training error and model complexity. If two classifiers have the same training error, the one with lower complexity is expected to generalise better.

For SVMs:

- small margin $\implies$ higher complexity $\implies$ higher generalisation error,
- large margin $\implies$ lower complexity $\implies$ lower generalisation error.

Why SVM is resistant to overfitting

SVMs can be resistant to overfitting because the decision boundary depends only on support vectors. Most training examples have $\lambda_i = 0$, they do not directly affect the final boundary. The model also explicitly maximises margin, which discourages overly sensitive boundaries.

However, SVMs can still overfit if:

- C is too large,
- the kernel is too flexible;
- the kernel parameters are poorly chosen;
- the data has high noise;
- the feature space is too complex relative to the number of examples.

So SVM is robust, but not magically immune to overfitting.

1.11 Comparison with Perceptrons and Neural Networks

Perceptron

A perceptron is simple and efficient. However, a basic perceptron can only learn linear decision boundaries. If the data is not linearly separable, the perceptron may fail to converge.

Backpropagation neural networks

Backpropagation neural networks can learn nonlinear decision boundaries. However, they can be difficult to train because:

- there may be many parameters;
- optimisation can be non-convex;
- training may get stuck in poor local minima;
- hyperparameter tuning can be difficult.

SVM

SVMs are relatively efficient and can learn nonlinear decision boundaries using kernels. The training problem is a QP, which has a global optimum.

1.12 Multiclass classification

SVM is naturally binary

Standard SVM is designed for binary classification $y^{(i)} \in \{-1, 1\}$. For multiclass problems, we need to reduce the problem to several binary classification problems.

Common strategies:

1. **One-vs-rest.** Train one classifier per class. For class k , examples from class k are treated as positive, and all other examples are treated as negative.
2. **One-vs-one.** Train one classifier for each pair of classes. For K classes, this gives

$$\frac{K(K-1)}{2}$$

binary classifiers.

2 Ensembles of Classifiers

2.1 Ensemble Learning

So far, we have mainly studied **single classifiers**. An ensemble classifier combines several classifiers to make one final prediction. Each individual classifier is called a **base classifier** or **ensemble member**.

Suppose we train M base classifiers $h_1, h_2, \dots, h_M$. For a new example $\mathbf{x}$, each classifier gives a prediction $h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_M(\mathbf{x})$. The ensemble combines these predictions, usually by **majority vote** or **weighted vote**. An ensemble can be understood as a **committee of classifiers**.

Majority voting

For classification, the most common combination rule is majority voting. For class c , count how many base classifiers predict c

$$\text{votes}(c) = \sum_{m=1}^M \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

The ensemble prediction is

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{m=1}^M \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

In majority voting, every base classifier has equal influence.

Weighted voting

In weighted voting, some classifiers are trusted more than others. Each base classifier has a weight α_m . The prediction is

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{m=1}^M \alpha_m \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

A more accurate classifier should receive a larger weight. This is the basic intuition behind boosting.

2.2 Why ensembles work

Two key conditions

An ensemble is useful when two conditions hold.

1. Each base classifier should be **reasonably accurate**. For binary classification, this means its error rate should be better than random guessing, $\epsilon < 0.5$.
2. The base classifiers should make **different mistakes**. In other words, their errors should not be perfectly correlated. Because, if all classifiers make exactly same mistakes, then voting does not help.

Example 2.2.1 (Independent errors). Suppose we have $M = 25$ binary base classifiers. Each has error rate $\epsilon = 0.35$. If all classifiers are identical, the ensemble error is still 0.35. But if their errors are independent, the ensemble makes a wrong prediction only when more than half of the classifiers (13 out of 25) are wrong:

$$\epsilon_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i}.$$

For $\epsilon = 0.35$, this is approximately $\epsilon_{\text{ensemble}} \approx 0.06$.

So the ensemble can be much better than each individual classifier.

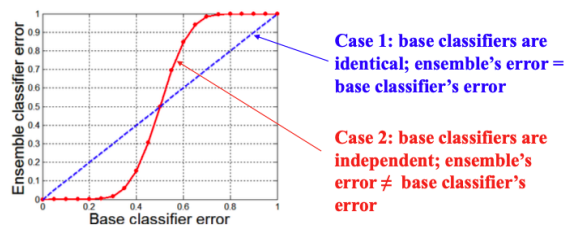


Figure 6: If $\epsilon < 0.5$, independent ensembles improve performance; if $\epsilon > 0.5$, the ensemble becomes worse.

2.3 Methods for creating Ensembles

General strategy

To create a useful ensemble, we want base classifiers that are both **accurate** and **diverse**. Diversity means the classifiers disagree in useful ways.

Common ways to create diversity include:

1. **Manipulating the training data.** Create different training sets from the original data. Examples include: **bagging**, **boosting**.
2. **Manipulating the attributes.** Train each classifier using a different subset of features. Example include Random Forest.
3. **Manipulating the learning algorithm.** Use the same data but change algorithm parameters, such as neural network architecture or initialisation.

2.4 Bagging

Bootstrap samples

Bagging stands for **bootstrap aggregation**. Given the training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, a bootstrap sample $\mathcal{D}_m$ is created by sampling n examples **with replacement** from $\mathcal{D}$.

Because sampling is with replacement:

- some examples appear multiple times;
- some examples do not appear at all;
- each bootstrap sample has size n .

On average, about 63% of the original examples appear at least once in a bootstrap sample.

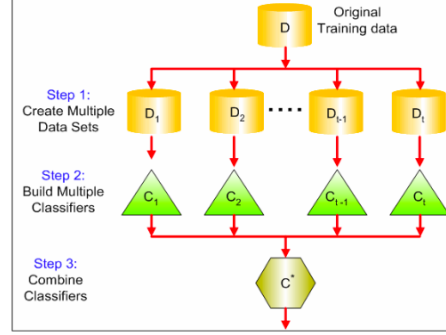


Figure 7: Original training data are sampled into several datasets, then several classifiers, then combined output.

Bagging algorithm

Bagging trains M classifiers as follows.

Algorithm 1 Bagging

Require: training set $\mathcal{D}$, integer M
for $m = 1, \dots, M$ **do**
 Create a bootstrap sample $\mathcal{D}_m$.
 Train a base classifier h_m on $\mathcal{D}_m$.
end for

To classify a new example $\mathbf{x}$, combine the predictions using majority vote:

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{m=1}^M \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

So bagging gives equal weight to all base classifiers.

When bagging is useful

Bagging is especially effective for **unstable classifiers**. An unstable classifier is one where small changes in the training data can lead to large changes in the learned model.

A decision tree is a typical unstable classifier. Bagging helps because each tree sees a slightly different bootstrap sample. The individual trees may vary, but their majority vote is more stable.

For regression, bagging combines predictions by averaging

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M h_m(\mathbf{x}).$$

Averaging multiple regression models can reduce mean squared error.

2.5 Boosting

Boosting builds classifiers sequentially. Unlike bagging, where base classifiers are trained independently, boosting trains each new classifier to focus on examples that previous classifiers found difficult.

Each training example has a weight $w_i \geq 0$. Initially, all examples have equal weight $w_i = 1/n$. After each classifier is trained:

- weights of misclassified examples increase;
- weights of correctly classified examples decrease.

So later classifiers focus more on difficult examples.

Boosting intuition

Boosting tries to make the base classifiers complement each other. The idea is that if classifier h_1 makes some mistakes, then classifier h_2 focuses more on those mistakes, and h_3 focuses on examples still difficult after h_1, h_2 . The final prediction is usually a weighted vote, where better classifiers receive larger voting weights.

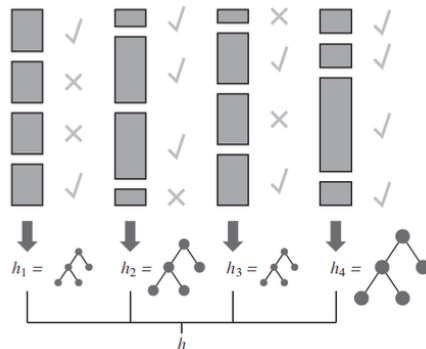


Figure 8: The rectangle height represents the example weight, and larger trees represent classifiers with larger influence.

2.6 AdaBoost

AdaBoost is the most popular boosting algorithm. It uses:

1. a probability distribution over training examples;
2. repeated training of base classifiers;
3. increased probability for misclassified examples;
4. weighted majority voting for the final prediction.

AdaBoost is commonly used with weak learners, such as decision stumps (one-level decision tree).

Training

Algorithm 2 AdaBoost Training

Require: training set $\mathcal{D}$, integer M

```

for  $i = 1, \dots, n$  do
     $w_i^{(1)} \leftarrow \frac{1}{n}$ 
end for
for  $m = 1, \dots, M$  do
    Train a classifier  $h_m$  using the current weights.
     $\epsilon_m \leftarrow \sum_{i=1}^n w_i^{(m)} \mathbf{1}\{h_m(\mathbf{x}^{(i)}) \neq y^{(i)}\}$ 
    Give more importance to misclassified examples,
    Normalise the example weights so they sum to 1.
end for

```

The exact update formulas vary by presentation, but the key idea is that misclassified examples get larger weights.

Prediction

The final AdaBoost classifier uses weighted voting:

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{m=1}^M \alpha_m \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

For binary classification with labels $\{-1, 1\}$, this is often written as

$$\hat{y} = \operatorname{sign} \left(\sum_{m=1}^M \alpha_m h_m(\mathbf{x}) \right).$$

A classifier with lower training error gets a larger α_m , so it has more influence in the final ensemble.

Example 2.6.1. Suppose the training set has $n = 10$ examples. Initially, $w_i = 0.1$ for $i = 1, \dots, 10$. If the first classifier misclassifies 3 examples, then after re-weighting:

- the 3 misclassified examples receive larger weights;
- the 7 correctly classified examples receive smaller weights.

The next classifier is therefore more likely to focus on the previously misclassified examples.

Boosting theorem

A weak learner is a classifier that performs slightly better than random guessing. For binary classification, $\epsilon < 0.5$.

Theorem 2.6.1 (Boosting theorem). *If the base learning algorithm is a weak learner, then AdaBoost can create an ensemble that classifies the training data perfectly, for sufficiently large M .*

This is why boosting can work very well with simple models such as decision stumps.

AdaBoost and overfitting

An interesting phenomenon is that as more base classifiers are added:

- training accuracy may reach 100%;
- test accuracy may still continue to improve.

This seems to contradict the usual expectation that more complex models overfit more easily. One explanation is that boosting continues to increase the confidence or margin of the ensemble predictions, not just fit the labels.

2.7 Gradient Boosting (Not Examinable)

Gradient Boosting is another important boosting method. Like AdaBoost, it builds an ensemble sequentially $h_1, h_2, \dots, h_M$. However, the key idea is different.

AdaBoost changes the weights of training examples so that later classifiers focus more on previous misclassified examples. Whereas Gradient Boosting builds each new model to correct the errors made by the current ensemble.

At round m , suppose the current ensemble is $F_{m-1}(\mathbf{x})$. Gradient Boosting adds a new model $h_m(\mathbf{x})$:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}),$$

where η is the learning rate. The purpose of h_m is to reduce the remaining prediction error.

Gradient Boosted Trees

In practice, Gradient Boosting most commonly uses decision trees as base learners. This gives **Gradient Boosted Trees** (GBT). The ensemble has the form of

$$F_M(\mathbf{x}) = \sum_{m=1}^M \eta h_m(\mathbf{x}),$$

where each h_m is a decision tree. Usually, the trees are shallow. For example, each tree may have only a small depth. This keeps each individual tree weak, while the ensemble becomes powerful by combining many trees.

GBTs are popular because they can model:

- nonlinear relationships;
- feature interactions;
- mixed numeric and categorical features;
- complex tabular data patterns.

They are especially strong for structured/tabular datasets.

The loss function view

Gradient Boosting is usually described as minimising a loss function. For a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, the goal is to minimise

$$\mathcal{L} = \sum_{i=1}^n \ell(y^{(i)}, F(\mathbf{x}^{(i)})),$$

where $F(\mathbf{x})$ is the ensemble prediction, ℓ is a loss function, and $\mathcal{L}$ is the total training loss.

For regression, a common loss is squared error

$$\ell(y, F(\mathbf{x})) = (y - F(\mathbf{x}))^2.$$

In this case, the next tree is fitted to the residual errors:

$$r^{(i)} = y^{(i)} - F_{m-1}(\mathbf{x}^{(i)}).$$

So for squared-error regression, Gradient Boosting is easy to understand: each new tree predicts the residuals of the previous ensemble.

XGBoost

Extreme Gradient Boosting (XGBoost) is a highly optimised and regularised implementation of GBTs. The main idea is still building trees sequentially to improve the current ensemble. But XGBoost adds many practical improvements, such as:

- regularisation to reduce overfitting;
- efficient tree construction;
- support for missing values;
- shrinkage / learning rate;
- column subsampling;
- row subsampling;
- parallelisation and system-level optimisation.

In practical machine learning, XGBoost is very popular for tabular data. It is often used in competitions and industry because it gives strong performance with careful tuning.

2.8 Bagging vs Boosting

Aspect	Bagging	Boosting
Training style	Parallel/independent	Sequential
Training data	Bootstrap samples	Re-weighted samples
Focus	Reduce instability	Focus on difficult examples
Voting	Equal vote	Weighted vote
Typical base learner	Decision trees	Weak learners, decision stumps
Main intuition	Average many noisy models	Build complementary experts

Table 1: Compact comparison between bagging and boosting.

2.9 Random Forest

Random Forest is an ensemble of decision trees. It combines two sources of randomness:

1. **Bagging.** Each tree is trained on a bootstrap sample.
2. **Random feature selection.** At each split, the tree only considers a random subset of features.

Algorithm 3 Random Forest

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, number of features considered at each split $k < d$, number of trees M

for $m = 1, \dots, M$ **do**
 Generate a bootstrap sample from $\mathcal{D}$.
 Grow a decision tree without pruning.
 At each split, randomly select k features.
 Choose the best split only among those k features.
end for

For classification, each tree predicts a class, and the forest uses majority vote:

$$\hat{y} = \operatorname{argmax}_c \sum_{m=1}^M \mathbf{1}\{h_m(\mathbf{x}) = c\}.$$

Why random feature selection helps

If every tree considers all features, different trees may become too similar. Random feature selection forces trees to explore different feature subsets. This reduces correlation between trees and increases diversity.

The performance of Random Forest depends on two factors:

1. strength of individual trees;
2. correlation between trees.

As k increases, each tree becomes stronger, but trees also become more correlated. So k controls a trade-off.

A rule of thumb is to choose $k = \log_2 d$.

Comparisons

Random Forest usually outperforms a single decision tree. It is often faster than AdaBoost and gives comparable accuracy. It is also relatively robust because:

- bagging reduces instability;
- random feature selection reduces correlation;
- many trees are averaged by majority vote.

Random Forest is one of the most widely used ensemble methods in practical machine learning.